

SPECGROVE: COORDINATING SPECULATIVE TREES FOR HIGH-THROUGHPUT DECODING

Anonymous authors

Paper under double-blind review

ABSTRACT

Concurrent block-diffusion speculative decoding must allocate shared verification capacity across requests that benefit differently from larger candidate trees. We introduce SpecGrove, which jointly selects tree sizes while reusing locally generated drafts. DFlash supplies parallel draft distributions, and a single DDTree expansion per request produces nested trees at several budgets without independently rebuilding each alternative. The draft probability mass of added prefixes estimates the benefit of expanding a tree. A dynamic program maximizes a priority-weighted estimate of token progress per measured verification cost for a *fixed admitted request set and fixed eligible tier set*; the guarantee is a finite-set optimum of this surrogate, not an end-to-end throughput optimum. Conditional bounds separate tier restriction, progress estimation, and cost estimation from the measured system objective. Selected trees share one target-model pass, with attention and cached states isolated across requests. A separate sampling analysis gives sufficient conditions for preserving the joint autoregressive output law and a perturbation bound when target conditionals differ; our BF16 implementation does not satisfy the exact-conditional premise in all audited traces. Experiments on Qwen3-4B and Qwen3-8B span four reasoning and code-generation benchmarks under closed-batch concurrency, with comparisons using the same verifier, precision, and timing protocol. SpecGrove improves end-to-end throughput, including online planning, by 13.0–32.1% over DFlash. Against a feasible fixed-45-node DDTree baseline, gains are 4.7–39.0% at the evaluated high-concurrency setting. Ablations attribute the largest consistent gain to adaptive tree sizing; equal and greedy adaptive-tier policies remain competitive with Bellman allocation. On Qwen3-8B mixed workloads, SPECGROVE exceeds matched TETRIS/ECHO-style allocation by 1.7–9.2% across four fixed-concurrency cells. In 256-request Poisson traces at a stable 2 requests/s, it matches their throughput while reducing p95 TTFT by 24.4–26.5% and p95 end-to-end latency by 30.6–33.8%; at an overloaded 4 requests/s, it improves median throughput by 1.7–2.2%, with tail changes that vary by arrival trace. Numerical audits and a separate quality study therefore bound the empirical claims rather than establishing implementation-level losslessness.

1 INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying several proposed continuations in parallel (Leviathan et al., 2023; Chen et al., 2023). Tree-based methods broaden the prefixes covered by one verification pass (Miao et al., 2024; Cai et al., 2024; Chen et al., 2024). DFlash generates block-diffusion draft distributions in one pass, and DDTree turns these distributions into a high-mass candidate tree (Chen et al., 2026; Ringel & Romano, 2026). These components make branching proposals inexpensive, but a fixed budget per request does not decide how a concurrent batch should share verification capacity.

Expanding one tree changes both its request’s coverage and the batch-wide verification cost. Uniformly large trees can inflate shared cost, while uniformly small trees discard useful coverage. Per-request progress therefore need not optimize aggregate progress per unit time.

Shared-budget speculation is already studied by batch schedulers such as TETRIS and ECHO (Wu et al., 2025; Hu et al., 2026); cross-request allocation is therefore not itself our novelty. SPECGROVE

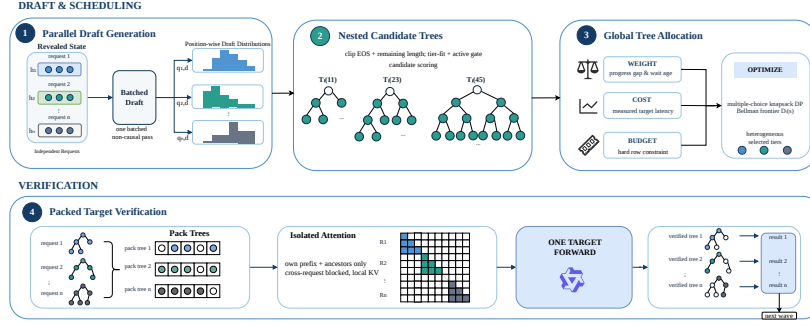


Figure 1: One SPECGROVE wave. Nested diffusion-draft tiers are valued by covered prefix mass and selected under a shared row budget and cost curve. Selected trees share one target pass. Each node sees its request’s cached prefix, its ancestors, and itself, but no unrelated branch or request. At every visited prefix, a fresh target sample is emitted; an off-tree sample ends the wave without another draw.

targets a narrower systems question for *branching block-diffusion drafts*: how to expose one DDTree construction as reusable capacity choices, value those choices, and pack heterogeneous trees without rebuilding independent candidates. It retains DDTree’s request-local expansion order, exposes nested prefixes as discrete tiers, and chooses one eligible tier per admitted request. Selected trees share one target pass with request-local ancestry and KV state (figure 1).

Covered prefix mass, service weights, and measured target cost define a per-wave surrogate rate. Bellman optimization yields the exact finite-set optimum of that surrogate for the *already admitted requests and eligible tiers*; it does not optimize admission, queueing delay, or full end-to-end time. Our analysis makes these boundaries explicit by separating tier, prediction, and execution-cost errors from measured throughput.

At $C = 8$, SPECGROVE exceeds feasible fixed-45-node DDTree by 4.7–39.0% at $R = 384$ and DFlash by 13.0–16.8% at $R = 193$. On Qwen3-8B mixed requests, it also exceeds matched TETRIS/ECHO-style controls in all four tested capacity cells, reduces stable-load tail latency, and raises overloaded throughput under Poisson arrivals. Timing includes scheduling; ablations and quality audits delineate the operating boundaries.

Our contributions are:

- **An executable tree-capacity interface.** Nested diffusion-draft tiers connect request-local prefix coverage to heterogeneous, request-isolated verification.
- **Scoped optimization guarantees.** A finite-set optimum and residual certificate characterize the surrogate allocator conditional on admission and eligible tiers; approximation bounds and a causal sequence-law theorem state the assumptions needed to connect it to progress and sampling.
- **Serving evidence across scheduler and arrival regimes.** Feasible DFlash and fixed-budget DDTree comparisons are complemented by matched TETRIS/ECHO-style controls and real-time Poisson arrivals; adaptive-tier ablations and quality audits identify where the gains arise and where simpler policies remain competitive.

2 BACKGROUND AND PROBLEM SETTING

Autoregressive and speculative generation. For a fixed revealed context x , a target model defines $p(y | x) = \prod_t p(y_t | x, y_{<t})$, with generation ending at EOS or a prescribed output cap. Conventional decoding evaluates one next-token conditional at a time. Speculative decoding uses a cheaper proposal to precompute several candidate continuations and evaluates their target conditionals together; an appropriate verification rule preserves the target law (Leviathan et al., 2023; Chen et al., 2023). Useful

progress must offset proposal and verification overhead. The tree traversal below uses fresh target sampling, rather than rejection-corrected chain sampling.

Block-diffusion drafts and candidate trees. DFlash uses target features to produce position-wise distributions q_d for a future block in one draft pass (Chen et al., 2026). They are not autoregressive conditionals on the tokens subsequently chosen within that block. For a continuation v , the product $Q(v) = \prod_{d=1}^{|v|} q_d(v_d)$ therefore defines a *draft surrogate*, whereas $P_p(v) = \prod_{d=1}^{|v|} p(v_d \mid x, v_{<d})$ is its target probability. DDTree expands high-mass prefixes in best-first order, yielding a prefix-closed tree under a node budget (Ringel & Romano, 2026). We reuse this construction rather than learn a new proposal model.

Verification, anchors, and progress. A tree’s root is an *anchor*: a target-sampled token already emitted, whose forward evaluation supplies the next conditional. Its descendants represent possible future prefixes. Tree attention preserves each node’s own autoregressive history (Miao et al., 2024; Ringel & Romano, 2026). Starting at the root, the verifier draws and emits a fresh target token. It follows a matching child, or ends the wave if that token is off-tree; EOS and the cap also stop traversal. There is no additional resampling on exit. For a depth- D tree T , without EOS or cap truncation,

$$\mu_p(T) = 1 + \sum_{v \in T \setminus \{\emptyset\}} P_p(v) \leq D + 1. \quad (1)$$

The one is a guaranteed *new* target draw, not the anchor. Width increases coverage, not maximum walk length; section 4.1 handles caps and EOS.

Concurrent verification. Let n be the admitted requests in a wave and R its packed-row budget; C denotes offered closed-batch concurrency. A full b -node non-root tier uses $b + 1$ rows, including its anchor. All requests share one pass, so total shape determines the modeled cost while progress remains request-specific. SPEC GROVE chooses tiers after admission; it does not replace the outer serving scheduler.

3 SPEC GROVE FRAMEWORK

3.1 WAVE STATE AND NESTED TREE TIERS

At a wave boundary, the allocator observes each admitted request’s generated length g_i , remaining output capacity $h_i \geq 1$, waiting age a_i , KV handle, and draft marginals. An outer controller ensures a jointly feasible admitted set. Decisions use revealed state and precede the upcoming target samples.

Let $\mathcal{B} = \{\beta_1 < \dots < \beta_K\}$ be the non-root budget labels. One batched DFlash pass and a canonical best-first expansion per request produce nested tiers $T_i(\beta_1) \subseteq \dots \subseteq T_i(\beta_K)$. EOS/length handling, capacity fitting, and expansion gating leave eligible sets $A_i \subseteq \mathcal{B}$, frozen for this wave. Upgrades reuse the same draft distributions and expansion order.

We distinguish a tier label from its realized shape. Let $m_i(b)$ be its retained non-root node count and $r_i(b)$ its charged verification rows. For an unpadded tier,

$$r_i(b) = 1 + m_i(b), \quad m_i(b) = b \text{ when no nodes are removed.} \quad (2)$$

Any padding is also charged to r_i ; only valid prefixes contribute utility. Reported full-tier feasibility uses $r_i(b) = b + 1$.

Define $\mathcal{V}_i(b) = \{v \in T_i(b) : 1 \leq |v| \leq h_i - 1, v \text{ contains no EOS}\}$. The score is

$$Q_i(v) = \prod_{d=1}^{|v|} q_{i,d}(v_d), \quad U_i(b) = 1 + \sum_{v \in \mathcal{V}_i(b)} Q_i(v). \quad (3)$$

It equals expected new-token progress only when the relevant draft and target prefix probabilities match. A nested upgrade adds the mass of its newly covered valid prefixes.

3.2 CAUSAL SERVICE WEIGHTS

Path mass alone can favor requests with concentrated draft distributions. To expose service state to the allocator, SPECROVE freezes a heuristic weight before target sampling; this is not a queueing-fairness or tail-latency guarantee:

$$w_i = 1 + \lambda \left[\frac{\max_j g_j - g_i}{M} + \frac{a_i}{\max(1, \max_j a_j)} \right], \quad (4)$$

where $M > 0$ is a common output cap, $0 \leq g_i \leq M$, $a_i \geq 0$, and $\lambda \geq 0$. The terms favor lagging or waiting requests. Hence $1 \leq w_i \leq 1 + 2\lambda$; weights alter capacity, not sampling. Appendix A.4 bounds the loss in unweighted surrogate rate without asserting a queueing-delay guarantee.

3.3 SHARED COST AND THE ALLOCATION OBJECTIVE

A positive lookup curve $C(s)$ models target-stage time at total packed rows s for each model/hardware configuration. It retains measured plateaus and jumps rather than imposing linear or convex cost. Row count is a calibrated approximation: request lengths and layout can affect realized time beyond s . Eligibility, utilities, weights, and the curve are fixed before solving the wave.

For allocation $\mathbf{b} = (b_1, \dots, b_n)$, define

$$\begin{aligned} s(\mathbf{b}) &= \sum_i r_i(b_i), & V(\mathbf{b}) &= \sum_i w_i U_i(b_i), \\ J(\mathbf{b}) &= \frac{V(\mathbf{b})}{C(s(\mathbf{b}))}, & b_i &\in A_i, \quad s(\mathbf{b}) \leq R. \end{aligned} \quad (5)$$

This *weighted surrogate service rate* trades expected path coverage against shared target-stage cost. Draft-target mismatch, service weighting, and omitted draft, construction, allocation, packing, and commit time distinguish it from end-to-end throughput. Experiments include these stages in complete wall time.

3.4 EXACT ALLOCATION OVER THE ELIGIBLE TIERS

For each exact row count s , let $D_i(s)$ be the maximum weighted utility attainable by the first i requests. The Bellman frontier is

$$\begin{aligned} D_0(0) &= 0, & D_0(s) &= -\infty \quad (s \neq 0), \\ D_i(s) &= \max_{\substack{b \in A_i \\ r_i(b) \leq s}} [D_{i-1}(s - r_i(b)) + w_i U_i(b)]. \end{aligned} \quad (6)$$

An empty maximum is $-\infty$. After processing all requests, the allocator maximizes $D_n(s)/C(s)$ over reachable $s \leq R$ and backtracks one tier per request. Theorem 1 proves why one numerator per exact row count is sufficient even for nonconvex cost.

Online work. The frontier is recomputed at each wave. With $k = \max_i |A_i|$, a back-pointer implementation requires $O(nRk)$ arithmetic operations, $O(R)$ reward memory, and $O(nR)$ back-pointer memory. The evaluated Python path copies allocation tuples and has a conservative $O(n^2 Rk)$ bound. Planning time is included in end-to-end throughput; its negligible latency or broader scaling is not assumed.

3.5 HETEROGENEOUS PACKED VERIFICATION

Selected trees carry request identifiers, depths, absolute positions, and parent indices. Each node attends only to its request’s cached prefix, tree ancestors, and itself; other branches and requests are masked. Request-local KV offsets isolate histories, and an anchor already cached must not be represented twice.

At each visited prefix, the verifier emits a fresh target draw. A matching child continues traversal; an off-tree draw, EOS, or the remaining cap stops it. Only the visited path is committed; other nodes are discarded. The exit token becomes the next anchor if generation continues. Numerical conditional equality remains separate from structural isolation (section 6.5).

Algorithm 1 One SPECROVE verification wave

Require: Admitted requests $\mathcal{I}$, revealed states, DFlash marginals, budgets $\mathcal{B}$, row cap R , cost curve C , coefficient λ

- 1: **for** each request $i \in \mathcal{I}$ **do**
- 2: Build one canonical best-first tree order from the draft marginals
- 3: Clip invalid paths; fit and gate tiers to obtain A_i
- 4: Compute $U_i(b)$ for $b \in A_i$ and freeze w_i
- 5: **end for**
- 6: Compute the exact-row frontier $D_n(s)$ using equation (6)
- 7: Choose reachable $s^* \leq R$ maximizing $D_n(s)/C(s)$
- 8: Backtrack the selected tiers $\mathbf{b}^*$
- 9: Pack trees with request-local ancestry, positions, and KV indices
- 10: Run one target forward pass
- 11: Ancestrally sample and commit target tokens; update request states

4 ANALYSIS: UTILITY, OPTIMALITY, AND OUTPUT LAW

All optimization statements condition on the admitted requests and fixed eligible tiers. Appendix A gives full proofs and separates surrogate guarantees from numerical execution.

4.1 EXPECTED PROGRESS IS COVERED PREFIX MASS

For request i , let D_i^{tree} be maximum tree depth, $H_i = \min(h_i, D_i^{\text{tree}} + 1)$, and $\mathcal{S}_{i,d}$ its non-EOS depth- d prefixes. A nonterminated request emits $N_i \geq 1$ tokens. Emitting token $d + 1$ requires its first d draws to follow one of the disjoint prefixes in $\mathcal{S}_{i,d}$. The tail-sum identity therefore gives

$$\mathbb{E}_p[N_i] = \sum_{d=0}^{H_i-1} \Pr(N_i \geq d+1) = \sum_{d=0}^{H_i-1} \sum_{v \in \mathcal{S}_{i,d}} P_p(v), \quad 1 \leq \mathbb{E}_p[N_i] \leq H_i. \quad (7)$$

Replacing P_p by Q_i yields equation (3), including its cap and EOS exclusions. If next-token kernels differ uniformly by at most ϵ in total variation, their utility difference is at most $H_i(H_i - 1)\epsilon/2$ (proposition 4). This is an assumption-based bound, not an error estimate from the finite audit.

4.2 EXACT ALLOCATION AND AN OPTIMALITY CERTIFICATE

Let $\mathcal{F}_R = \{\mathbf{b} \in \prod_i A_i : s(\mathbf{b}) \leq R\}$ and $\mathcal{S}_R = \{s(\mathbf{b}) : \mathbf{b} \in \mathcal{F}_R\}$. The following result uses the standard fractional-programming residual (Dinkelbach, 1967) with the exact-row frontier.

Theorem 1 (Exact tier allocation and residual certificate). *Suppose $\mathcal{F}_R$ is finite and nonempty, r_i are positive integers, and $C(s) > 0$ on $\mathcal{S}_R$. For fixed additive utilities and weights, equation (6) returns*

$$J^* = \max_{s \in \mathcal{S}_R} \frac{D_n(s)}{C(s)}, \quad \Phi(\rho) = \max_{s \in \mathcal{S}_R} \{D_n(s) - \rho C(s)\}. \quad (8)$$

Let $c_{\min}$ and $c_{\max}$ be the minimum and maximum of C on $\mathcal{S}_R$. Every feasible $\mathbf{b}$ satisfies

$$0 \leq \frac{\Phi(J(\mathbf{b}))}{c_{\max}} \leq J^* - J(\mathbf{b}) \leq \frac{\Phi(J(\mathbf{b}))}{c_{\min}}. \quad (9)$$

In particular, $\Phi(J(\mathbf{b})) = 0$ if and only if $\mathbf{b}$ is surrogate-optimal.

Proof. Partition exact- s allocations by their last tier. Induction in equation (6) gives the largest numerator for each s ; its denominator is common and positive. Scanning all reachable s proves the optimum. Also $\Phi(\rho) = \max_{\mathbf{a} \in \mathcal{F}_R} C(s(\mathbf{a}))[J(\mathbf{a}) - \rho]$. At $\rho = J(\mathbf{b}) \leq J^*$, evaluating an optimizer bounds this maximum below by $c_{\min}(J^* - \rho)$, while every term is at most $c_{\max}(J^* - \rho)$. Rearrangement proves the certificate. $\square$

This is global optimality *over the fixed eligible tiers*, not all trees or serving schedules. A beneficial upgrade satisfies $C\Delta V > V\Delta C$ when both denominators are positive. Consequently, the optimum need not fill every row. Full certificates require recorded input utilities, costs, and frontiers; their availability for all experimental waves is not assumed.

4.3 APPROXIMATION AND FULL-TIME COSTS

Corollary 2 (Tier restriction and estimation loss). *Let $F = V/C$ be a nonnegative reference ratio on a finite set $\mathcal{F}$, with $C \geq c_{\min}^{\mathcal{F}} > 0$. Let $\emptyset \neq \mathcal{G} \subseteq \mathcal{F}$. Suppose a projection $\mathcal{P} : \mathcal{F} \rightarrow \mathcal{G}$ does not increase reference cost and loses at most $L \geq 0$ utility. If $|\hat{F} - F| \leq \eta$ on $\mathcal{G}$ and $\hat{a} \in \mathcal{G}$ is within $\zeta \geq 0$ of its estimated optimum, then*

$$0 \leq F_{\mathcal{F}}^* - F(\hat{a}) \leq L/c_{\min}^{\mathcal{F}} + 2\eta + \zeta. \quad (10)$$

Proof. Projection of a reference optimizer bounds $F_{\mathcal{F}}^* - F_{\mathcal{G}}^*$ by $L/c_{\min}^{\mathcal{F}}$. Comparing estimated and reference ratios at $\hat{a}$ and a maximizer on $\mathcal{G}$ adds at most $2\eta + \zeta$; see appendix A.3. $\square$

Uniform errors $|\hat{V} - V| \leq \epsilon_V$, $|\hat{C} - C| \leq \epsilon_C$, with $V \leq V_{\max}$, $C \geq c_{\min} > 0$ and $\hat{C} \geq \hat{c}_{\min} > 0$, imply

$$\eta = \frac{\epsilon_V}{\hat{c}_{\min}} + \frac{V_{\max}\epsilon_C}{c_{\min}\hat{c}_{\min}}. \quad (11)$$

Fewer rows alone do not ensure the projection’s cost condition on a nonmonotone curve. The bounds are not a numerical throughput certificate without uniform errors for the desired metric.

Execution overhead. If the old full time is $C + H_0$ and an upgrade changes it to $C + \Delta C + H_1$, then its full-time ratio improves exactly when $(C + H_0)\Delta V > V(\Delta C + H_1 - H_0)$, assuming positive denominators. Both baseline and new overhead matter. Appendix A.6 gives a conditional long-run decomposition; the experiments use actual total tokens divided by complete wall time.

4.4 CAUSAL SCHEDULING AND NUMERICAL EXACTNESS

For a fixed collection of requests, assume exact same-prefix target conditionals, causal decisions using no unrevealed target randomness, fresh independent target draws, and eventual completion at each request’s EOS or fixed cap. For completed outputs (including EOS when emitted), coupling to independent reference streams gives

$$\Pr(Y_1 = y_1, \dots, Y_n = y_n) = \prod_i \prod_{t=1}^{|y_i|} p_i(y_{i,t} \mid y_{i,<t}). \quad (12)$$

The schedule changes which factor is revealed next, not its value (theorem 7). The perturbation bound in equation (21) separately requires uniform conditional-error bounds. Observed BF16 posterior differences do not supply such a certificate, so implemented task quality is evaluated separately.

5 EXPERIMENTAL METHODOLOGY

Models, workloads, and execution. We evaluate Qwen3-4B and Qwen3-8B (Yang et al., 2025) on one NVIDIA H20 using 128 frozen prompts from each of GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021), HumanEval (Chen et al., 2021), and sanitized MBPP (Austin et al., 2021). The target uses eager BF16/SDPA; proposal probabilities and posterior bookkeeping use FP32. CUDA graphs and compiler-only acceleration are disabled for the common-framework comparisons. The principal matrix uses closed batches at fixed concurrency. A Qwen3-8B serving study adds matched scheduler controls and real-time Poisson arrivals.

Allocation and measurement. The deployed budgets are $\mathcal{B} = \{11, 23, 45\}$ non-root nodes, with $\lambda = 1$, capacity fitting, and larger-tier expansion suppressed above eight active requests. Each model uses its measured verification-cost curve. The main matrix uses temperature one, output cap 256, seeds 17/29/43, $R = 193$, and $C \in \{1, 4, 8, 16\}$. Budget sweeps additionally use $R \in \{96, 384\}$; mixed workloads interleave datasets and prompt lengths and extend to $C = 32$. Throughput is total emitted tokens divided by complete wall time, including allocation. Reported intervals use 2,000 paired prompt-batch bootstrap resamples with prompt-level clustering. The separate quality study uses cap 1,024 and five seeds at $C = 4$ (section 6.5).

Matched schedulers and open-loop serving. The scheduler study holds the DFlash proposal, nested DDTree candidates, verifier, row cap, request seeds, precision, and timing boundaries fixed. The TETRIS-style control globally ranks ancestor-closed tier increments by added draft covered mass per row (Wu et al., 2025). The ECHO-style control applies sparse confidence gates at tier boundaries, then spends residual capacity on coverage expansion (Hu et al., 2026); its thresholds are draft-only medians from a disjoint 16-prompt slice. These controls transfer the papers’ allocation principles to DDTree tiers rather than reproduce their original serving stacks. We test $(R, C) \in \{(193, 8), (193, 16), (384, 16), (384, 32)\}$ on 128 balanced mixed requests with three seeds. The open-loop study uses 256 balanced mixed requests, $R = 384$, at most 32 active requests, and independent Poisson traces at 1, 2, and 4 requests/s. Arrivals occur in wall-clock time; all schedulers use request-local cache repacking. We report queue delay, time to first token (TTFT), time per output token (TPOT), end-to-end latency, completed-request rate, token throughput, fairness, and online planning time.

Comparators and capacity. AR uses the same target decoder. Matched DFlash and DDTree share the verifier, batching, precision, and timing boundaries; native official runs at $C = 1$ retain their original code paths. At $R = 193$, the offered DFlash shape fits through $C = 8$, whereas fixed 45-node DDTree fits through $C = 4$. A dash denotes only that this *specified fixed shape* exceeds the prescribed per-wave row budget; it is not evidence that DDTree as a method cannot serve the workload. The adaptive-size ablations compare Bellman allocation with equal and greedy policies over the same DDTree-derived tiers (section 6.4 and table 9). The new TETRIS/ECHO-style controls isolate scheduler policy on this backend; they do not support a claim against either original end-to-end system. Appendix B reports the complete matrices and protocol counts.

6 RESULTS

6.1 MATCHED THROUGHPUT AT SHARED CAPACITY

At $R = 193$, table 1 summarizes concurrent performance; table 3 gives all main cells and AR intervals. Against matched DFlash, SPECGROVE improves end-to-end throughput by 13.0–32.1% in all 24 combinations at $C \in \{1, 4, 8\}$. This range includes isolated requests, not just allocation gains. At $C = 8$, both methods fit: gains are 13.0–16.8% across all eight model–task combinations, with 2.70–4.11 $\times$ AR throughput. Fixed-45-node DDTree exceeds this row budget.

Concurrent trees with both methods feasible. At $R = 384, C = 8$, DDTree’s full tiers use $8(45 + 1) = 368$ rows. Table 1(b) shows SPECGROVE’s gains of 4.7–19.1% on 4B and 10.0–39.0% on 8B, positive in all eight combinations. Tables 5 and 6 retain the complete sweeps. This compares a matched fixed-45-node baseline, not all tuned DDTree budgets; the reported ratios are point estimates rather than significance tests.

Fixed trees and isolated requests. At $C = 4$, matched DDTree fits. SPECGROVE’s changes range from 0.3% to 14.4% on 8B and -4.8% to $+0.6\%$ on 4B. At $C = 1$, it is within 0.6% of matched DDTree, but native DDTree is 8–11% faster (table 7). Isolated requests offer no cross-request allocation opportunity; the native implementation gap is not attributed to an unmeasured component.

Feasibility is distinct from a speed comparison. At $C = 16$ and $R = 193$, SPECGROVE delivers 1.74–3.34 $\times$ AR, but the evaluated fixed baselines exceed capacity. This is not a speedup over absent configurations. Sixteen minimum tiers consume 192 rows, leaving little upgrade freedom; feasibility alone does not establish informed allocation’s value.

6.2 MIXED WORKLOADS AND LARGER BUDGETS

The mixed suite tests requests with different datasets and prompt lengths (table 2). At $R = 193$ and $C \in \{4, 8\}$, SPECGROVE exceeds feasible DFlash by 20.7–24.8% on 4B and 17.4–25.7% on 8B. At $R = 384$, DFlash also fits at $C = 16$: SPECGROVE reaches 906.4 and 781.4 tokens/s, improving throughput by 16.1% and 14.5%, respectively. Both DFlash and SPECGROVE fit in this high-concurrency comparison.

Table 1: Concurrent throughput and serving. Panels (a,b) report the main natural-task matrix; panels (c,d) compare matched schedulers on Qwen3-8B mixed requests. T/E denotes TETRIS-style/ECHO-style.

(a) SPEC-GROVE end-to-end throughput (tok/s)

Model	Data	$C = 1$	$C = 4$	$C = 8$	$C = 16$
4B	GSM8K	182.6	488.1	762.6	978.6
	MATH	184.2	481.1	728.2	963.0
	HumanEval	186.2	519.4	762.1	912.5
	MBPP	159.2	284.4	357.0	468.7
8B	GSM8K	186.0	485.3	753.0	861.1
	MATH	193.4	505.1	708.4	834.9
	HumanEval	193.8	517.6	711.6	803.9
	MBPP	150.1	279.1	383.6	514.2

(b) Capacity-feasible matched comparisons

Model	Data	/AR ($\times$) (193, 8)	vs DFlash (%) (193, 8)	vs DDTree (%) (193, 4)	vs DDTree (%) (384, 8)
4B	GSM8K	3.19	+16.6	-4.8	+14.5
	MATH	2.92	+16.4	-4.3	+16.2
	HumanEval	3.05	+13.0	-2.0	+19.1
	MBPP	3.83	+16.8	+0.6	+4.7
8B	GSM8K	2.95	+15.0	+7.2	+39.0
	MATH	2.70	+15.7	+10.3	+36.1
	HumanEval	2.70	+16.7	+14.4	+36.5
	MBPP	4.11	+16.5	+0.3	+10.0

(c) Fixed-concurrency scheduler control (tok/s)

R	C	SPEC-GROVE	TETRIS-style	ECHO-style	gain T/E
193	8	650.7	595.7	595.9	+9.2/+9.2%
193	16	772.6	751.8	751.4	+1.7/+1.9%
384	16	764.8	717.6	719.7	+6.8/+6.6%
384	32	826.1	809.8	810.3	+2.0/+2.1%

(d) Poisson serving: SPEC-GROVE and paired changes vs. T/E

Req/s	tok/s	gain T/E	TTFT ₉₅ (ms)	reduction T/E	E2E ₉₅ (ms)	reduction T/E
1	192.5	-0.3/+0.2%	67.4	+3.6/+5.5%	2049	+7.8/+8.1%
2	370.4	+0.5/+0.0%	84.0	+24.4/+26.5%	2633	+30.6/+33.8%
4	682.3	+2.2/+1.7%	4778	+12.4/+8.2%	16702	+10.8/+6.8%

Panels (a,b) use 128 prompts/task; panels (c,d) use balanced mixed requests. All report three seeds. Panels (c,d) show medians, and gain/reduction entries are paired per-seed medians. The 4-req/s traces are overloaded; their tail changes have mixed signs across seeds. Tables 3 and 4 give intervals and complete method-wise metrics.

Against feasible DDTree configurations, the mixed-workload changes span -4.1% to +9.2% on 4B and +6.4% to +29.5% on 8B. At $C = 32$, SPEC-GROVE reaches 1,003.2 and 844.7 tokens/s, retaining $1.64\times$ and $1.48\times$ AR.

6.3 MATCHED SCHEDULERS AND POISSON ARRIVALS

The controls choose among identical DDTree tiers, isolating allocation from proposal coverage. SPEC-GROVE wins all 12 seed-capacity comparisons against each control: paired median gains span 1.7–9.2%, while Bellman planning p95 remains 0.070–0.139 ms per wave.

At the arrival-limited rate of 1 request/s, paired throughput changes are $-0.3\%/+0.2\%$. At the stable 2-request/s load, SPEC-GROVE changes throughput by $+0.5\%/+0.0\%$, while reducing p95 TTFT by 24.4%/26.5% and p95 end-to-end latency by 30.6%/33.8% versus TETRIS/ECHO-style scheduling. At 4 requests/s, all methods complete only 3.41–3.45 requests/s and p95 queueing reaches seconds: this is an overload stress test, not a steady-state latency point. SPEC-GROVE improves paired median throughput by 2.2%/1.7%; median TTFT reductions are 12.4%/8.2%, but the per-seed ranges include -35.0% and -113.6% . Its Jain index, 0.660, lies between TETRIS-style’s 0.649 and ECHO-style’s 0.713 (table 4).

6.4 WHAT THE COMPONENT COMPARISONS ESTABLISH

Adaptive tiers improve geometric-mean throughput over fixed 11-node trees by 9.6%/8.5% (4B/8B), request-aware selection adds 3.6%/5.5% over random tiers, and the measured cost curve adds 1.4%/2.4% over linear cost. Equal and greedy adaptive-tier policies remain within 0.9% of SPEC-GROVE (table 9). Thus adaptive sizing supplies the clearest gain; the present three-tier experiments do not establish Bellman optimization as uniquely necessary.

6.5 TASK QUALITY AND NUMERICAL AGREEMENT

At $C = 4$, the separate five-seed quality study supports a two-point noninferiority margin against AR in five of eight settings; Qwen3-4B MBPP differs by -2.03 points with paired 95% interval $[-3.75, -0.31]$ (table 8). Request-isolation checks pass, but exact numerical agreement does not: greedy sequences match AR in 1,248/1,536 (4B) and 1,155/1,536 (8B) repeated executions, and sampled same-prefix posteriors differ at temperature one. The exact-conditional theorem therefore does not certify the BF16 kernels (appendix B.9).

7 RELATED WORK

Drafting and tree construction. Classical speculative decoding corrects cheap proposals with target probabilities (Leviathan et al., 2023; Chen et al., 2023); SpecInfer, Medusa, EAGLE, and Sequoia explore trees or multiple proposals (Miao et al., 2024; Cai et al., 2024; Li et al., 2024; Chen et al., 2024). We reuse DFlash marginals and DDTree construction (Chen et al., 2026; Ringel & Romano, 2026). Unlike single-tree CaDDTree (Zhang et al., 2026), SPEC-GROVE jointly assigns request-specific tiers under a shared cost.

Batch scheduling and serving. BASS optimizes batched execution; TETRIS selects draft tokens across requests; D-Cut adapts depth; and ECHO uses sparse confidence gates (Qian et al., 2024; Wu et al., 2025; Liu et al., 2026; Hu et al., 2026). SPEC-GROVE contributes a DFlash–DDTree interface for reusable nested tiers and isolated heterogeneous verification. Our style controls compare allocation rules inside that interface; reproducing the original systems also requires their proposal models and runtimes. Other proposers must expose reusable prefix-closed choices, progress scores, and compatible costs. Orca, vLLM, and SGLang address complementary admission, memory, and reuse decisions (Yu et al., 2022; Kwon et al., 2023; Zheng et al., 2024).

8 DISCUSSION AND CONCLUSION

SPEC-GROVE turns one local DDTree expansion into reusable tiers, allocates those tiers across requests, and verifies the selected trees in one isolated pass. Matched fixed-batch and Poisson experiments support this interface; competitive equal/greedy policies show that exact optimization is not always needed. The evaluation covers one H20, with scheduler and arrival studies limited to Qwen3-8B mixed requests. Native TETRIS/ECHO systems, multi-hour workloads, planning beyond $C = 32$, and other hardware remain to be tested. Cost recalibration, overload queueing variability, and BF16 posterior discrepancies delimit deployment claims.

REFERENCES

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pp. 5209–5235, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable and robust speculative decoding. In *Advances in Neural Information Processing Systems 37*, 2024. doi: 10.52202/079017-4116.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Werner Dinkelbach. On nonlinear fractional programming. *Management Science*, 13(7):492–498, 1967. doi: 10.1287/mnsc.13.7.492.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS 2021 Datasets and Benchmarks Track*, volume 1, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Xinyi Hu, Yuhao Shen, Baolin Zhang, Hengxin Zhang, Jun Dai, Shuang Ge, Lei Chen, Yue Li, and Mingcheng Wan. ECHO: Elastic speculative decoding with sparse gating for high-concurrency scenarios, 2026. URL <https://arxiv.org/abs/2604.09603>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *PMLR*, pp. 19274–19286, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pp. 28935–28948, 2024. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- Tianyu Liu, Yuhao Shen, Rui Cen, Junhan Shi, Jiebin Zhang, Guangshuo Qin, Hong Liu, Song Liu, Guanghua Yu, and Jianchen Zhu. D-Cut: Adaptive verification depth pruning for batched speculative decoding, 2026. URL <https://arxiv.org/abs/2607.14647>.

- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 932–949. ACM, 2024. doi: 10.1145/3620666.3651335. URL <https://doi.org/10.1145/3620666.3651335>.
- Haifeng Qian, Sujun Kumar Gonugondla, Sungsoo Ha, Mingyue Shang, Sanjay Krishna Gouda, Ramesh Nallapati, Sudipta Sengupta, Xiaofei Ma, and Anoop Deoras. BASS: Batched attention-optimized speculative sampling. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 8214–8224. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.489. URL <https://aclanthology.org/2024.findings-acl.489/>.
- Liran Ringel and Yaniv Romano. Accelerating speculative decoding with block diffusion draft trees, 2026. URL <https://arxiv.org/abs/2604.12989>.
- Zhaoxuan Wu, Zijian Zhou, Arun Verma, Alok Prakash, Daniela Rus, and Bryan Kian Hsiang Low. TETRIS: Optimal draft token selection for batch speculative decoding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33329–33345. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1598. URL <https://aclanthology.org/2025.acl-long.1598/>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pp. 521–538. USENIX Association, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Shuai Zhang, Huachuan Qiu, Hongliang He, and Yong Dai. Cost-aware diffusion draft trees for speculative decoding, 2026. URL <https://arxiv.org/abs/2606.01813>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37*, 2024. doi: 10.52202/079017-2000.

AI USE STATEMENT

Generative AI tools assisted literature retrieval, manuscript organization and language editing, interpretation of reported results, and LaTeX, citation, and consistency checks. AI-assisted scripts were used to check finite mathematical examples and reported-table consistency during revision; these checks do not constitute an independent reproduction of the GPU experiments. The authors are responsible for reviewing all AI-assisted content and for the final technical claims, experimental reporting, and citations.

REPRODUCIBILITY STATEMENT

Section 5 and appendix B specify the reported settings and complete result matrices; appendix A provides assumptions and proofs. Independent replication additionally requires exact checkpoint/tokenizer revisions, frozen prompt identifiers, evaluation scripts, calibrated cost tables, and per-prompt timing/quality records.

ETHICS STATEMENT

Faster inference does not remove model biases, harmful outputs, or privacy risks. Numerical and task-quality differences require consideration before deployment. Throughput alone does not establish energy or carbon savings.

A THEORETICAL STATEMENTS AND PROOFS

This appendix distinguishes progress under target kernels, optimization of a finite surrogate, and preservation of the output law. Bounds involving prediction or numerical error require uniform assumptions on the stated domain; they are not empirical error estimates for the evaluated implementation.

A.1 EXPECTED PROGRESS WITH EOS AND AN OUTPUT CAP

Fix a nonterminated request, a finite prefix-closed candidate tree T with distinct sibling labels and maximum depth D , and $h \geq 1$ remaining output positions. Verification samples the target kernel at the current prefix and follows the corresponding tree edge when one exists. The wave stops after an off-tree token, EOS, or the output cap; the terminating sampled token is emitted. Let N count newly emitted tokens. Write $\mathcal{S}_d(T)$ for the depth- d tree prefixes containing no EOS, and $P_p(v)$ for their target probabilities, with $P_p(\emptyset) = 1$.

Proposition 3 (Exact wave utility). *The expected progress is*

$$\mathbb{E}_p[N] = \sum_{d=0}^{\min(D, h-1)} \sum_{v \in \mathcal{S}_d(T)} P_p(v), \quad 1 \leq \mathbb{E}_p[N] \leq \min(h, D+1). \quad (13)$$

If no tree path contains EOS and $h \geq D+1$, this expression reduces to equation (1).

Proof. The tail-sum identity gives $\mathbb{E}[N] = \sum_{k=1}^h \Pr(N \geq k)$. Emitting the k th token requires the first $k-1$ samples to follow a non-EOS path in T . Sibling labels are distinct, so different prefixes at depth $k-1$ define disjoint events. Consequently,

$$\Pr(N \geq k) = \sum_{v \in \mathcal{S}_{k-1}(T)} P_p(v)$$

for $k-1 \leq D$, and is zero otherwise. Substitution proves the identity. The depth-zero sum is one and every depth sum is at most one, giving the stated bounds. Without clipping or EOS, all tree nodes enter the sum. $\square$

Node count controls the number of covered prefixes, not the number of tokens on a realized walk. In particular, width cannot increase progress beyond $D+1$. Substituting draft masses Q for target probabilities gives the implementable utility after excluding EOS-containing paths and nodes deeper than $h-1$.

Let $\mu_p(T, h)$ denote equation (13). For another family of next-token kernels $q(\cdot | v)$, define $\mu_q(T, h)$ using the same tree and stopping rule. Position-wise diffusion marginals induce such a reference family by taking $q(\cdot | v) = q_{|v|+1}(\cdot)$; this does not assume they equal target conditionals. We use $\text{TV}(p, q) = \frac{1}{2} \sum_z |p(z) - q(z)|$.

Proposition 4 (Draft-to-target utility error). *Suppose $0 \leq \epsilon \leq 1$ and $\text{TV}(p(\cdot | v), q(\cdot | v)) \leq \epsilon$ at every relevant nonterminal prefix. For $H = \min(h, D+1)$,*

$$|\mu_p(T, h) - \mu_q(T, h)| \leq \sum_{d=1}^{H-1} [1 - (1 - \epsilon)^d] \leq \frac{H(H-1)}{2} \epsilon. \quad (14)$$

Proof. Couple the two traversals maximally at each next-token draw, sending either process to an absorbing stop state on tree exit or EOS. While their nonterminal prefixes agree, the next tokens agree with probability at least $1 - \epsilon$; after a common stop, agreement persists. The stopped processes agree through step d with probability at least $(1 - \epsilon)^d$. Agreement implies the same membership in the depth- d non-EOS tree event. The difference between the two event probabilities is thus at most $1 - (1 - \epsilon)^d$. Summing over the nonzero depths in equation (13) proves the first bound. The second follows from $1 - (1 - \epsilon)^d \leq d\epsilon$. $\square$

A.2 FINITE-SET OPTIMALITY AND A CHECKABLE CERTIFICATE

Let $\mathcal{B} = \{\beta_1 < \dots < \beta_K\}$ be a finite ordered budget set. Each admitted request i has a nonempty eligible set $A_i \subseteq \mathcal{B}$ after clipping, gating, and capacity fitting. Define

$$\mathcal{F}_R = \left\{ \mathbf{b} \in \prod_i A_i : \sum_i r_i(b_i) \leq R \right\}, \quad \mathcal{S}_R = \{s(\mathbf{b}) : \mathbf{b} \in \mathcal{F}_R\}.$$

Assume $\mathcal{F}_R \neq \emptyset$ and $c_{\min} = \min_{\mathbf{b} \in \mathcal{F}_R} C(s(\mathbf{b})) > 0$.

Detailed proof of theorem 1. The costs $r_i(b)$ are the fixed positive integer row charges, including any retained padding. No choice-dependent interaction is permitted in U_i , w_i , or A_i beyond the declared total-row constraint; any such interaction would require additional Bellman state. The proof uses exact arithmetic on these fixed inputs.

Proof. We prove by induction that $D_i(s)$ is the maximum weighted utility attainable by the first i requests using exactly s rows. Initialization establishes the claim for $i = 0$. An exact- s allocation for i requests selects some $b \in A_i$ for its final request and leaves $s - r_i(b)$ rows for the others. By the induction hypothesis, its utility cannot exceed the corresponding term of the Bellman recurrence. Conversely, a maximizing final tier together with an induction-achieving allocation attains that term. Finiteness ensures that each reachable maximum is attained.

At fixed s , all allocations have the same positive denominator $C(s)$, so maximizing the numerator also maximizes the ratio. Evaluating every reachable $s \leq R$ therefore covers the entire feasible set and returns its largest surrogate rate. $\square$

The frontier also defines a residual certificate. Let

$$\begin{aligned} \Phi(\rho) &= \max_{\mathbf{b} \in \mathcal{F}_R} \{V(\mathbf{b}) - \rho C(s(\mathbf{b}))\} \\ &= \max_{s \in \mathcal{S}_R} \{D_n(s) - \rho C(s)\}. \end{aligned} \tag{15}$$

As the maximum of finitely many affine functions, Φ is continuous. For $\rho_2 > \rho_1$, take a maximizer $\mathbf{b}_2$ at ρ_2 . Then

$$\Phi(\rho_2) \leq \Phi(\rho_1) - c_{\min}(\rho_2 - \rho_1) < \Phi(\rho_1).$$

Thus Φ is strictly decreasing. Writing a term as $C(s(\mathbf{b}))[J(\mathbf{b}) - \rho]$ shows that its unique zero is J^* . Set $c_{\max} = \max_{s \in \mathcal{S}_R} C(s)$. Any feasible allocation satisfies

$$0 \leq \frac{\Phi(J(\mathbf{b}))}{c_{\max}} \leq J^* - J(\mathbf{b}) \leq \frac{\Phi(J(\mathbf{b}))}{c_{\min}}. \tag{16}$$

Indeed, for an optimizer $\mathbf{b}^*$,

$$\Phi(J(\mathbf{b})) \geq C(s(\mathbf{b}^*)) [J^* - J(\mathbf{b})] \geq c_{\min} [J^* - J(\mathbf{b})].$$

For the other inequality, every feasible $\mathbf{a}$ has $C(s(\mathbf{a}))[J(\mathbf{a}) - J(\mathbf{b})] \leq c_{\max} [J^* - J(\mathbf{b})]$: negative terms are harmless, and nonnegative terms satisfy both bounds factorwise. Maximizing over $\mathbf{a}$ proves the lower gap bound. This also proves that zero residual is equivalent to optimality.

The reported traces contain selected allocations and row counts. Applying this certificate to a wave additionally requires its input utilities, costs, complete frontier, and back-pointers. The mathematical certificate should not be confused with a claim that these full records were collected for every reported wave.

A.3 TIER RESTRICTION, PREDICTION ERROR, AND OVERHEAD

Theorem 5 (Tier restriction). *Let $\mathcal{F}$ be a finite nonempty action set with $V(a) \geq 0$ and $C(s(a)) > 0$. Define $J(a) = V(a)/C(s(a))$, $J_{\mathcal{H}}^* = \max_{a \in \mathcal{H}} J(a)$ for nonempty $\mathcal{H} \subseteq \mathcal{F}$, and $c_{\min}^{\mathcal{F}} = \min_{a \in \mathcal{F}} C(s(a))$. Suppose $\emptyset \neq \mathcal{G} \subseteq \mathcal{F}$ and a map $\mathcal{P} : \mathcal{F} \rightarrow \mathcal{G}$ satisfies, for some $L \geq 0$ and every $a \in \mathcal{F}$,*

$$C(s(\mathcal{P}a)) \leq C(s(a)), \quad V(\mathcal{P}a) \geq \max\{0, V(a) - L\}.$$

Then

$$0 \leq J_{\mathcal{F}}^* - J_{\mathcal{G}}^* \leq L/c_{\min}^{\mathcal{F}}. \tag{17}$$

Proof. Set inclusion gives the lower bound. Let a^* maximize J over $\mathcal{F}$. If $V(a^*) \geq L$, the projection assumptions give

$$J_{\mathcal{G}}^* \geq J(\mathcal{P}a^*) \geq \frac{V(a^*) - L}{C(s(a^*))} \geq J_{\mathcal{F}}^* - L/c_{\min}^{\mathcal{F}}.$$

If $V(a^*) < L$, then $J_{\mathcal{F}}^* < L/c_{\min}^{\mathcal{F}}$ and $J_{\mathcal{G}}^* \geq 0$, which yields the same conclusion. $\square$

For nested integer-budget trees, let $\tilde{A}_i$ be a richer finite candidate set and $A_i \subseteq \tilde{A}_i$ the retained set. Let $\mathcal{F}$ and $\mathcal{G}$ be their row-feasible product sets. Assume $A_i \cap (-\infty, b]$ is nonempty for every $b \in \tilde{A}_i$, weights are nonnegative, utilities and row charges are nondecreasing with budget, and C is nondecreasing in total rows. Define

$$\pi_i(b) = \max\{a \in A_i : a \leq b\}, \quad L = \sum_i w_i \max_{b \in \tilde{A}_i} [U_i(b) - U_i(\pi_i(b))].$$

Componentwise projection preserves feasibility, does not increase cost, and loses at most L weighted utility. Theorem 5 therefore applies. The monotonic-cost assumption is sufficient for this particular projection, not required by the Bellman optimizer itself. A numerical tier-loss certificate would require evaluating L for the deployed eligible sets.

Theorem 6 (Uniform prediction error). *Let $F = V/C$ and $\hat{F} = \hat{V}/\hat{C}$ be reference and estimated ratios on a common finite nonempty set $\mathcal{A}$. Suppose $\epsilon_V, \epsilon_C \geq 0$, $0 \leq V \leq V_{\max}$, $C \geq c_{\min} > 0$, $\hat{C} \geq \hat{c}_{\min} > 0$, and $|\hat{V} - V| \leq \epsilon_V$, $|\hat{C} - C| \leq \epsilon_C$ uniformly on $\mathcal{A}$. Then*

$$|\hat{F} - F| \leq \eta = \frac{\epsilon_V}{\hat{c}_{\min}} + \frac{V_{\max}\epsilon_C}{c_{\min}\hat{c}_{\min}}.$$

If $\zeta \geq 0$ and $\hat{F}(\hat{a}) \geq \max_{a \in \mathcal{A}} \hat{F}(a) - \zeta$, then $F(\hat{a}) \geq F^ - 2\eta - \zeta$. If $\zeta = 0$ and $\hat{F}(\hat{a}) - \hat{F}(a) > 2\eta$ for all $a \neq \hat{a}$, then $\hat{a}$ uniquely maximizes F .*

Proof. Subtract the ratios:

$$\hat{F} - F = \frac{\hat{V} - V}{\hat{C}} + \frac{V(C - \hat{C})}{C\hat{C}}.$$

The triangle inequality and denominator lower bounds give $|\hat{F} - F| \leq \eta$. For a reference maximizer a^* ,

$$F(\hat{a}) \geq \hat{F}(\hat{a}) - \eta \geq \hat{F}(a^*) - \zeta - \eta \geq F(a^*) - 2\eta - \zeta.$$

For uniqueness, every alternative a satisfies $F(\hat{a}) - F(a) \geq \hat{F}(\hat{a}) - \hat{F}(a) - 2\eta > 0$. $\square$

These statements bound ratios on a specified action set. Transferring them to end-to-end throughput requires the reference numerator and denominator to represent the desired output metric and full execution time, together with the stated uniform error estimates.

Proof of corollary 2. Choose $a^* \in \arg \max_{a \in \mathcal{F}} F(a)$ and $a_{\mathcal{G}}^* \in \arg \max_{a \in \mathcal{G}} F(a)$. By theorem 5, $F_{\mathcal{F}}^* - F_{\mathcal{G}}^* \leq L/c_{\min}^{\mathcal{F}}$. Uniform ratio error and ζ -optimality imply

$$F(\hat{a}) \geq \hat{F}(\hat{a}) - \eta \geq \hat{F}(a_{\mathcal{G}}^*) - \zeta - \eta \geq F_{\mathcal{G}}^* - \zeta - 2\eta.$$

Adding these two inequalities proves equation (10); $\hat{a} \in \mathcal{F}$ gives its lower bound. For an exactly solved estimated frontier, $\zeta = 0$. A positive ζ may instead account for an approximate solve. The result does not estimate any of L, η, ζ from the reported throughput tables.

The cost of an upgrade. For $V, C > 0$ and $C + \Delta C > 0$, cross-multiplication gives

$$\frac{V + \Delta V}{C + \Delta C} > \frac{V}{C} \iff C\Delta V > V\Delta C. \quad (18)$$

For full execution time, let the old non-target overhead be $H_0 \geq 0$ and the new overhead $H_1 \geq 0$. With $V > 0$ and both full-time denominators positive,

$$\frac{V + \Delta V}{C + \Delta C + H_1} > \frac{V}{C + H_0} \iff (C + H_0)\Delta V > V(\Delta C + H_1 - H_0). \quad (19)$$

Indeed, multiplying the positive denominators and cancelling $V(C + H_0)$ gives the right-hand inequality. The incremental overhead $H_1 - H_0$ may be negative. The special case $H_0 = 0, H_1 = H$ reduces to $H < C\Delta V/V - \Delta C$; it must not be used as a general comparison when the baseline also has overhead. Before extra overhead, $\Delta C > 0$ requires the marginal utility rate to exceed the old average rate, while $\Delta C = 0$ makes every positive utility increment beneficial. None of these conditions measures implementation latency.

A.4 SERVICE WEIGHTS AND UNWEIGHTED PROGRESS

On a common finite nonempty action set with positive cost and nonnegative utilities, assume $0 < w_{\min} \leq w_i \leq w_{\max}$. Let a_w maximize the weighted ratio and a_0 maximize the unweighted rate $F_0(a) = \sum_i U_i(a_i)/C(s(a))$. Then

$$F_0(a_w) \geq \frac{w_{\min}}{w_{\max}} F_0(a_0). \quad (20)$$

The weight sandwich gives

$$w_{\max} F_0(a_w) \geq J_w(a_w) \geq J_w(a_0) \geq w_{\min} F_0(a_0).$$

Dividing proves the claim. Under equation (4), the factor becomes $1/(1 + 2\lambda)$. This is a one-wave surrogate comparison; queueing-delay or fairness statements additionally require an arrival and admission model.

A.5 JOINT SEQUENCE LAW AND NUMERICAL PERTURBATIONS

Theorem 7 (Causal joint sequence law). *Consider finitely many requests with reference autoregressive kernels p_i , each decoded until EOS or a fixed finite request-local cap. Assume:*

1. *every visited target conditional equals p_i at the same prefix;*
2. *scheduling and tree construction depend only on revealed histories and draft randomness independent of unrevealed target randomness;*
3. *target samples use fresh independent randomness across requests and emitted positions, every emitted token is a target sample, and unverified draft tokens are never emitted;*
4. *every request eventually reaches its prescribed stopping position.*

Then any such shared-budget schedule produces

$$\mathcal{L}(Y_1, \dots, Y_n) = \bigotimes_i \mathcal{L}_{p_i}(Y_i).$$

Proof. Construct independent reference streams on one probability space. For each request, independent uniforms $Z_{i,t}$ and inverse transforms of $p_i(\cdot \mid Y_{i,<t})$ generate its AR sequence up to EOS or the cap; EOS is absorbing.

The scheduler retains revealed prefixes and a next-token cursor for each request. Causality ensures that it selects the request, tree, and budget before reading the uniform at that cursor. Exact conditional equality allows its target draw to use the same $Z_{i,t}$, producing the next reference token. If the token follows a tree edge, verification continues. If it leaves the tree or terminates the request, the wave stops, but the emitted token still agrees with the reference. Random variables for unvisited candidate nodes do not affect the output.

Induction over emitted tokens shows that the shared execution reveals prefixes of the independent reference streams without altering them. Rebuilding trees and reallocating capacity change only when and how many tokens are revealed. Eventual completion reveals each reference sequence through its prescribed stopping point, establishing the joint product law. $\square$

Theorem 8 (Posterior perturbation). *For an output process of at most M tokens, suppose $0 \leq \epsilon_t \leq 1$ and, at every common revealed prefix at position t ,*

$$\text{TV}(p_t(\cdot \mid y_{<t}), \tilde{p}_t(\cdot \mid y_{<t})) \leq \epsilon_t.$$

With EOS padded by an absorbing token,

$$\text{TV}(\mathcal{L}_p(Y), \mathcal{L}_{\tilde{p}}(Y)) \leq 1 - \prod_{t=1}^M (1 - \epsilon_t) \leq \sum_{t=1}^M \epsilon_t. \quad (21)$$

For every score $f(Y) \in [0, 1]$, the difference in expected score is bounded by the same quantity.

Proof. Conditioned on agreement so far, maximal coupling makes the next samples agree with probability at least $1 - \epsilon_t$. Agreement of the entire padded sequence therefore has probability at least $\prod_t (1 - \epsilon_t)$. The coupling inequality proves the first bound, and the union bound gives the second. The expectation difference of a score in $[0, 1]$ is at most total variation, for example by integrating its level sets. $\square$

If an implemented conditional also depends on a packed layout or other requests, the bound must hold uniformly over those states, not only the local token prefix. Conditional mixtures then obey the same bound by convexity of total variation. The perturbation result requires bounds over all relevant common histories. Finite sampled traces inspect only observed histories and do not supply this uniform certificate. Passing structural mask/KV checks, matching some sampled sequences, and obtaining a noninferiority decision are distinct observations; none alone proves exact equality of target conditionals.

A.6 FROM WAVE PROGRESS TO MEASURED THROUGHPUT

Assume an i.i.d. regenerative-wave model with emitted-token count N_t , full duration $T_t > 0$ almost surely, $0 < \mathbb{E}[T_t] < \infty$, and $\mathbb{E}[N_t] < \infty$. The strong law yields

$$\frac{\sum_{t=1}^m N_t}{\sum_{t=1}^m T_t} \xrightarrow{\text{a.s.}} \frac{\mathbb{E}[N_t]}{\mathbb{E}[T_t]}.$$

For two such systems $j \in \{0, 1\}$, write $\bar{T}_j = \mathbb{E}[T^{(j)}] = \bar{T}_{\text{target},j} + \bar{T}_{\text{other},j}$. Assume $\bar{T}_j > 0$, $\bar{T}_{\text{target},j} > 0$, and $\mathbb{E}[N^{(j)}] > 0$. Define

$$\begin{aligned} \alpha &= \frac{\bar{T}_{\text{target},0}}{\bar{T}_0}, & \gamma &= \frac{\bar{T}_{\text{target},0}}{\bar{T}_{\text{target},1}}, \\ \delta &= \frac{\bar{T}_{\text{other},1} - \bar{T}_{\text{other},0}}{\bar{T}_0}, & a &= \frac{\mathbb{E}[N^{(1)}]}{\mathbb{E}[N^{(0)}]}. \end{aligned}$$

The ratio of long-run token rates is

$$S = \frac{a}{1 - \alpha + \alpha/\gamma + \delta}. \quad (22)$$

To verify this identity, divide system 1's time decomposition by $\bar{T}_0$:

$$\frac{\bar{T}_1}{\bar{T}_0} = 1 - \alpha + \alpha/\gamma + \delta.$$

Substitution into $(\mathbb{E}[N^{(1)}]/\bar{T}_1)/(\mathbb{E}[N^{(0)}]/\bar{T}_0)$ proves the result.

The empirical metric uses the same ratio-of-totals form, total emitted tokens divided by total complete wall time. The convergence statement above is conditional on the regenerative assumptions; evolving closed-batch states are not asserted to form i.i.d. waves. In particular, the decomposition explains how other-stage costs can offset a target-stage gain without proving that maximizing the per-wave surrogate maximizes the measured batch throughput.

B EXPERIMENTAL PROTOCOL AND COMPLETE RESULTS

B.1 PROTOCOL, COUNTS, AND INTERPRETATION

The frozen experiment record reports completion of every planned phase. Per model, it contains 2,208 main-throughput groups (8,448 method executions), 640 quality groups (2,560 executions), 1,096

correctness groups (4,160 executions), 1,024 native-baseline groups (6,144 executions), 4,368 budget groups (16,512 executions), 1,104 ablation groups (14,208 executions), and 348 mixed-workload groups (1,248 executions): 10,788 groups and 53,280 executions in total. The Qwen3-8B scheduler supplement adds 108 matched closed-batch groups (324 method executions) and 27 final 256-request Poisson traces. These are protocol counts, not additional independent benchmark tasks.

The main and ablation studies use temperature one, cap 256, seeds 17/29/43, and 128 frozen prompts per dataset. The quality study uses cap 1,024, concurrency four, and seeds 17/29/43/67/101. Accuracy and code pass rates are aggregated over prompt clusters. Reported confidence intervals are prompt-cluster bootstrap intervals for the specified dataset-comparator pair, not simultaneous family-wise intervals. The quality noninferiority (NI) decision uses the pre-specified one-sided two-percentage-point margin; it is reported separately from the displayed paired 95% interval.

Every throughput ratio below compares total emitted tokens per complete wall-clock time. The denominator includes online planning rather than reporting only target-forward latency. The original matrix does not separately log planning time; the scheduler supplement records it per wave and reports p95 values. A dash indicates that the specified fixed shape exceeds the row budget for that offered concurrency; it is neither a measured zero throughput nor an impossibility result for the baseline method. The tables retain the reported numeric precision. MATH denotes the MATH-500 evaluation subset throughout.

B.2 COMPLETE MIXED-WORKLOAD MATRIX

Table 2: Mixed-request throughput. The AR column is a speedup ratio; DFlash and DDTree columns are relative gains (%). Bold marks $C \geq 16$ throughput/AR results and comparator gains of at least 1%; dashes denote fixed configurations exceeding the prescribed per-wave row cap.

Model	R	C	SPECGROVE tok/s	/AR ($\times$)	vs DFlash	vs DDTree
4B	193	4	433.5	4.250	+24.8	-3.8
	193	8	675.3	3.408	+20.7	—
	193	16	895.6	2.361	—	—
	384	4	433.5	4.265	+24.8	-4.1
	384	8	661.1	3.339	+18.4	+9.2
	384	16	906.4	2.361	+16.1	—
	384	32	1003.2	1.644	—	—
	193	4	439.3	4.194	+25.7	+7.1
	193	8	652.8	3.203	+17.4	—
	193	16	781.3	2.023	—	—
	384	4	436.5	4.188	+25.7	+6.4
	384	8	643.1	3.182	+16.4	+29.5
8B	384	16	781.4	2.037	+14.5	—
	384	32	844.7	1.480	—	—

B.3 COMPLETE MAIN THROUGHPUT MATRIX

The 32 rows span two models, four tasks, and four concurrency levels at $R = 193$. DFlash is feasible in the 24 rows at $C \in \{1, 4, 8\}$, while 45-node DDTree is feasible at $C \in \{1, 4\}$. The $C = 8$ rows therefore support matched AR/DFlash comparisons, not a comparison with an unmeasured DDTree configuration.

Table 3: Complete row-budget-193 main matrix. The /AR column reports point speedup with paired prompt-batch bootstrap 95% interval; other columns are throughput ratios. Bold marks $C \geq 8$ throughput/AR results and comparator ratios of at least 1.01; dashes mark infeasible baselines.

Model	Data	C	tok/s	/AR [95% CI]	/DFlash	/DDTree
4B	GSM8K	1	182.6	4.566 [4.444, 4.694]	1.265	0.999
4B	GSM8K	4	488.1	4.019 [3.886, 4.163]	1.233	0.952
4B	GSM8K	8	762.6	3.185 [3.071, 3.311]	1.166	–
4B	GSM8K	16	978.6	2.123 [2.052, 2.203]	–	–
4B	MATH	1	184.2	4.638 [4.451, 4.824]	1.253	0.996
4B	MATH	4	481.1	3.751 [3.598, 3.929]	1.227	0.957
4B	MATH	8	728.2	2.923 [2.829, 3.023]	1.164	–
4B	MATH	16	963.0	2.049 [1.997, 2.100]	–	–
4B	HumanEval	1	186.2	4.734 [4.600, 4.879]	1.286	1.001
4B	HumanEval	4	519.4	4.061 [3.890, 4.215]	1.262	0.980
4B	HumanEval	8	762.1	3.055 [2.924, 3.202]	1.130	–
4B	HumanEval	16	912.5	1.971 [1.886, 2.052]	–	–
4B	MBPP	1	159.2	4.024 [3.799, 4.247]	1.195	0.999
4B	MBPP	4	284.4	4.364 [3.961, 4.768]	1.184	1.006
4B	MBPP	8	357.0	3.826 [3.431, 4.199]	1.168	–
4B	MBPP	16	468.7	3.172 [2.790, 3.550]	–	–
8B	GSM8K	1	186.0	4.614 [4.483, 4.741]	1.299	1.001
8B	GSM8K	4	485.3	3.910 [3.749, 4.086]	1.215	1.072
8B	GSM8K	8	753.0	2.946 [2.820, 3.081]	1.150	–
8B	GSM8K	16	861.1	1.785 [1.701, 1.870]	–	–
8B	MATH	1	193.4	4.591 [4.406, 4.769]	1.291	0.998
8B	MATH	4	505.1	3.712 [3.567, 3.886]	1.226	1.103
8B	MATH	8	708.4	2.701 [2.626, 2.779]	1.157	–
8B	MATH	16	834.9	1.774 [1.701, 1.847]	–	–
8B	HumanEval	1	193.8	4.643 [4.498, 4.795]	1.321	0.998
8B	HumanEval	4	517.6	3.822 [3.679, 3.973]	1.230	1.144
8B	HumanEval	8	711.6	2.697 [2.604, 2.801]	1.167	–
8B	HumanEval	16	803.9	1.737 [1.680, 1.796]	–	–
8B	MBPP	1	150.1	4.117 [3.861, 4.377]	1.184	0.994
8B	MBPP	4	279.1	4.451 [3.979, 4.969]	1.178	1.003
8B	MBPP	8	383.6	4.109 [3.615, 4.642]	1.165	–
8B	MBPP	16	514.2	3.342 [2.928, 3.812]	–	–

B.4 COMPLETE OPEN-LOOP SERVING METRICS

Each entry is the median of three complete 256-request traces. The same arrival times, request seeds, prompt order, active-request cap, and request-local cache implementation are used by all three methods within a seed. Queue is admission time minus arrival time; TTFT and end-to-end latency are measured from arrival. Planning is the allocator’s per-wave p95 latency.

Table 4: Real-time Poisson serving on Qwen3-8B. All latency columns are milliseconds; J is Jain’s index over per-request service rates.

Req/s	Method	tok/s	done/s	Queue ₉₅	TTFT ₉₅	TPOT ₉₅	E2E ₉₅	J
1	SPECGROVE	192.5	0.978	37.3	67.4	8.5	2049	0.940
	TETRIS-style	193.2	0.978	39.6	70.2	9.6	2222	0.931
	ECHO-style	192.1	0.978	40.6	70.0	9.8	2198	0.926
2	SPECGROVE	370.4	1.874	52.3	84.0	11.1	2633	0.917
	TETRIS-style	366.7	1.870	75.0	111.1	15.5	3795	0.860
	ECHO-style	365.2	1.869	70.1	110.0	16.1	3979	0.869
4	SPECGROVE	682.3	3.449	4708	4778	61.0	16702	0.660
	TETRIS-style	671.1	3.418	5707	5738	59.8	18746	0.649
	ECHO-style	669.9	3.410	5234	5316	60.0	18029	0.713

Done/s is completed requests divided by finite-trace wall time, so it can differ from the nominal Poisson rate. The 4-req/s point is overloaded and its tail changes vary across traces. J uses per-request output tokens divided by end-to-end latency. Median allocator p95 time for SPECGROVE is 0.054/0.110/0.141 ms at 1/2/4 requests/s; TETRIS-style uses 0.023/0.030/0.053 ms and ECHO-style 0.027/0.036/0.054 ms. Bold marks the best value within each arrival rate.

B.5 SENSITIVITY TO THE ROW BUDGET

The following sweeps retain the same task and model distinctions while varying the row cap. Budget 96 admits fewer fixed-shape configurations; budget 384 supports comparisons at larger concurrency. These are capacity sweeps within the evaluated implementation, not cross-hardware measurements.

Table 5: Complete 4B budget sweep at $R \in \{96, 384\}$. Entries /baseline are throughput ratios. Bold marks $C \geq 16$ throughput/AR results and comparator ratios of at least 1.01.

R	Data	C	tok/s	/AR	/DFlash	/DDTree
96	GSM8K	1	186.6	4.560	1.264	0.998
96	GSM8K	4	505.5	4.038	1.236	–
96	GSM8K	8	751.1	3.064	–	–
96	MATH	1	188.6	4.640	1.255	0.997
96	MATH	4	497.2	3.773	1.223	–
96	MATH	8	747.7	2.918	–	–
96	HumanEval	1	183.9	4.814	1.287	1.017
96	HumanEval	4	515.5	4.113	1.267	–
96	HumanEval	8	746.2	3.056	–	–
96	MBPP	1	153.5	4.016	1.193	0.992
96	MBPP	4	274.8	4.353	1.179	–
96	MBPP	8	362.0	4.016	–	–
384	GSM8K	1	186.0	4.539	1.263	0.998
384	GSM8K	4	502.6	4.010	1.233	0.958
384	GSM8K	8	758.7	3.097	1.136	1.145
384	GSM8K	16	999.1	2.139	1.142	–
384	GSM8K	32	1059.8	1.353	–	–
384	MATH	1	179.2	4.457	1.203	0.953
384	MATH	4	485.3	3.768	1.223	0.956
384	MATH	8	740.3	2.991	1.183	1.162
384	MATH	16	951.0	2.071	1.143	–
384	MATH	32	1005.5	1.521	–	–
384	HumanEval	1	181.8	4.786	1.288	1.000
384	HumanEval	4	515.6	4.113	1.261	0.978
384	HumanEval	8	763.5	3.058	1.121	1.191
384	HumanEval	16	916.1	1.967	1.129	–
384	HumanEval	32	950.9	1.469	–	–
384	MBPP	1	152.4	4.022	1.193	0.992
384	MBPP	4	278.5	4.398	1.196	1.011
384	MBPP	8	367.6	3.993	1.208	1.047
384	MBPP	16	470.9	3.263	1.144	–
384	MBPP	32	583.2	2.554	–	–

Table 6: Complete 8B budget sweep at $R \in \{96, 384\}$. Entries /baseline are throughput ratios. Bold marks $C \geq 16$ throughput/AR results and comparator ratios of at least 1.01.

R	Data	C	tok/s	/AR	/DFlash	/DDTree
96	GSM8K	1	192.4	4.591	1.296	1.000
96	GSM8K	4	518.5	4.006	1.238	–
96	GSM8K	8	746.7	2.946	–	–
96	MATH	1	190.0	4.536	1.288	0.999
96	MATH	4	499.0	3.714	1.234	–
96	MATH	8	714.7	2.749	–	–
96	HumanEval	1	192.3	4.650	1.317	1.000
96	HumanEval	4	516.0	3.872	1.241	–
96	HumanEval	8	712.9	2.771	–	–
96	MBPP	1	163.1	4.280	1.188	1.003
96	MBPP	4	306.6	4.551	1.181	–
96	MBPP	8	382.6	3.925	–	–
384	GSM8K	1	192.3	4.562	1.296	1.000
384	GSM8K	4	503.1	3.908	1.210	1.090
384	GSM8K	8	735.9	2.939	1.141	1.390
384	GSM8K	16	866.3	1.827	1.152	–
384	GSM8K	32	907.9	1.213	–	–
384	MATH	1	189.5	4.546	1.286	1.000
384	MATH	4	498.5	3.702	1.225	1.096
384	MATH	8	710.8	2.720	1.167	1.361
384	MATH	16	838.9	1.783	1.163	–
384	MATH	32	880.3	1.431	–	–
384	HumanEval	1	183.3	4.637	1.313	0.995
384	HumanEval	4	482.1	3.885	1.233	1.111
384	HumanEval	8	672.5	2.806	1.150	1.365
384	HumanEval	16	771.2	1.760	1.165	–
384	HumanEval	32	792.6	1.327	–	–
384	MBPP	1	162.5	4.330	1.198	0.998
384	MBPP	4	304.2	4.570	1.183	1.016
384	MBPP	8	394.7	4.036	1.135	1.100
384	MBPP	16	515.2	3.379	1.153	–
384	MBPP	32	596.8	2.779	–	–

B.6 NATIVE SINGLE-REQUEST IMPLEMENTATIONS

These comparisons preserve the official baselines’ native execution paths rather than sharing SPEC-GROVE’s verifier. They measure an implementation-level end-to-end gap at $C = 1$ and should not be conflated with the common-framework DDTree ratios in table 3. No component-level timing decomposition is inferred from the native gap.

Table 7: Native single-request comparison at row budget 193. Bold marks ratios of at least 1.01 in favor of SPEC-GROVE.

Model	Data	SPEC-GROVE tok/s	/native DFlash	/native DDTree
4B	GSM8K	186.5	1.175	0.929
4B	MATH	190.6	1.158	0.917
4B	HumanEval	193.5	1.182	0.919
4B	MBPP	163.5	1.095	0.914
8B	GSM8K	186.0	1.210	0.911
8B	MATH	188.0	1.202	0.923
8B	HumanEval	189.1	1.195	0.900
8B	MBPP	175.6	1.078	0.915

B.7 TASK-QUALITY RESULTS

The quality experiment compares SPEC-GROVE with AR at a longer output cap than the throughput study. Five of eight settings support the pre-specified noninferiority margin. A “no” decision means that this margin was not established; it is not equivalent to proving a loss exceeding that margin. The 4B MBPP point difference and interval are reported without attributing them to a particular numerical mechanism.

Table 8: Complete quality comparison to AR. NI is the pre-specified one-sided 2-point noninferiority decision. Bold marks NI-supported decisions; the direction and interval of every quality difference are shown.

Model	Data	AR (%)	SPEC-GROVE (%)	Δ pp	paired 95% CI	NI
4B	GSM8K	90.62	91.88	+1.25	[-0.16, +2.66]	yes
4B	MATH	70.62	71.56	+0.94	[-1.88, +3.75]	yes
4B	HumanEval	83.91	84.22	+0.31	[-2.19, +2.81]	yes
4B	MBPP	67.03	65.00	-2.03	[-3.75, -0.31]	no
8B	GSM8K	92.97	92.50	-0.47	[-2.50, +1.72]	no
8B	MATH	72.97	73.91	+0.94	[-1.56, +3.59]	yes
8B	HumanEval	87.03	85.94	-1.09	[-4.06, +1.88]	no
8B	MBPP	70.62	70.31	-0.31	[-2.19, +1.72]	yes

B.8 COMPLETE COMPONENT ABLATIONS

Each reported gain is SPEC-GROVE’s geometric-mean end-to-end throughput change relative to one ablated policy on GSM8K and MATH-500 across $C \in \{1, 4, 8, 16\}$. Fixed 45-node trees fit only at $C = 1$ and $C = 4$, giving four comparable cells per model. These ablations are separate controlled contrasts, so their gains should not be added.

Table 9: Aggregate component ablations. Positive gain favors the complete SPEC-GROVE policy. Bold marks gains of at least 0.5% and wins in at least 6/8 cells.

Ablated policy	4B gain	wins	8B gain	wins
fixed 11-node trees	+9.6%	8/8	+8.5%	8/8
random tier allocation	+3.6%	8/8	+5.5%	8/8
linear cost model	+1.4%	7/8	+2.4%	8/8
constant cost model	-1.3%	2/8	+3.9%	6/8
no service weights	+0.5%	6/8	-0.1%	3/8
no active gate	-0.0%	3/8	-0.2%	3/8
no tier fitting	+0.3%	7/8	-0.3%	3/8
equal allocation	-0.9%	4/8	-0.3%	1/8
greedy marginal allocation	+0.0%	6/8	-0.6%	1/8
fixed 45-node trees	-2.5%	0/4	+2.6%	2/4
age-only admission	-0.1%	4/8	-0.2%	4/8

Gain is the geometric-mean end-to-end throughput change of SPEC-GROVE relative to the named ablated policy across GSM8K/MATH-500 and $C \in \{1, 4, 8, 16\}$. Fixed 45-node trees are feasible in four cells per model. Wins count cells with a positive change; each row is a separate comparison.

Reading the mixed outcomes. Adaptive sizing and informed selection outperform fixed-small and random-tier controls in every evaluated cell; the measured cost curve improves over linear cost in 15/16. Equal and greedy policies operate on the same adaptive DDTree-derived tiers and remain competitive, while service weights, gating, and tier fitting have mixed signs. Thus the ablation supports adaptive sizing more strongly than it supports Bellman DP as a uniquely necessary optimizer. The small eligible action sets and the difference between surrogate and full-time objectives further limit solver-level claims. The results identify useful components without establishing dominance of every design choice in every regime.

B.9 STRUCTURAL AND NUMERICAL DIAGNOSTICS

The structural audit passes checks of request-isolated ancestry masks, position metadata, and KV bookkeeping. Numerical agreement is tested separately. At temperature zero, generated sequences match AR in 1,248 of 1,536 repeated condition executions for Qwen3-4B and 1,155 of 1,536 for Qwen3-8B. These denominators count repeated condition executions, not independent prompt samples. At temperature one, direct same-prefix posterior comparisons reveal numerical differences. The experiment record consequently marks `strict_sequence_law_certified=false`.

These diagnostics delimit the application of theorem 7: structural isolation is checked, but exact equality of the implemented and reference conditionals is not established. They do not provide a uniform total-variation bound over reachable histories, demonstrate a repair, or isolate the cause of any task-quality difference. The theorem, perturbation bound, and quality study answer complementary questions rather than serving as interchangeable correctness certificates.